Master: You can write grains using a Python function. In fact, you can write public functions in Python to implement custom grains on the respective minions. Salt will execute all the 'public' functions found in the modules located in the grains package or the custom grains directory. The function must return a python *dict*, where the keys in the *dict* are the names of the grains and the values are the values of the grains.

Custom grains should be placed in a *_grains* directory located under the file *_roots* specified by the master *config* file. The default path is */srv/salt/_grains*. Custom grains will be distributed to the minions when *state.highstate* is run, or by executing the *saltutil.sync_grains* or *saltutil.sync_all* functions.

```
/srv/salt/_grains/customGrains.py
#!/usr/bin/env python
def yourfunction():
     # initialize a grains dictionary
     grains = {}
     # Some code for logic that sets grains like
     grains['datacenter']='datacenter4'
     grains['environment']='test'
     grains['department']='HR'
     grains['role']='webserver'
     return grains
```

Core grains can be overridden by custom grains. As there are several ways of defining custom grains, there is an order of precedence which should be kept in mind when defining them. The order of evaluation is as follows.

Each successive evaluation overrides the previous ones, so any grains defined in */etc/salt/grains* that have the same name as a core grain will override that core grain. Similarly, */etc/salt/minion* overrides both core grains and grains set in */etc/salt/grains*, and custom grain modules will override *any* grains of the same name.

Matching grains in a top file

Grains can be used in the *.sls* file and the top file. For example:

```
'role:webserver':
    - match: grain
    - tomcat

'role:postgres':
    - match: grain
    - database
```

Pillars

A pillar is an interface that generates and stores highly sensitive data specific to a particular minion, such as cryptographic keys and passwords. It stores data in a key/value pair and the data is managed in a similar way as the Salt State Tree. The Salt Master server maintains a *pillar_roots* setup that matches the structure of the *file_roots* used in the Salt file server.

The configuration for the *pillar_roots* in the master *config* file is identical in behaviour and function to *file_roots*. To start setting up the pillar, create the */srv/pillar* directory and uncomment *pillar_roots* in the */etc/salt/master configuration* file.

```
pillar_roots:
  base:
    - /srv/pillar
```

Top files

Similar to the state tree, the pillar comprises of *.sls* files and has a top file.

```
/srv/pillar/top.sls
base:
   '*':
     - pkgs
```

In the above top file, pillar data in the *pkgs* file is available to all the minions. Like Salt states, the *pkgs* pillar can be located at */srv/pillar/pkgs.sls* or */srv/pillar/pkgs/init.sls* and it follows the same namespace as the Salt state.

```
/srv/pillar/pkgs/init.sls
pkgs:
  - pam_ldap
- vim
```

Grains within a pillar

Grains can be used in the pillar's top file as well as pillar *.sls* files, and it is useful to deliver specific Salt pillar data to minions with different properties. For example:

```
base:
  '*':
    - packages

  'role:webserver':
    - match: grain
    - webserver

  'os:redhat':
    - match: grain
    - rhnpackages
```

Viewing pillar data

Once the *pillar* is set up, the data can be viewed on the minion via the pillar module, which comes with functions, *pillar.items* and *pillar.raw*. *Pillar.items* will return a freshly reloaded pillar and *pillar.raw* will return the current pillar without a refresh:

```
salt '*' pillar.items
```